

SimplexLP

Revised Simplex and Interior Point Methods for Linear Programming Project Implementation Report

Šehzada Sijarić (19964)

Project #13

1 Introduction

SimplexLP is a desktop application that solves linear programs with two different methods and draws what each of them actually does. The first is the revised simplex method, which walks along the boundary of the feasible region from vertex to vertex. The second is a primal-dual interior point method, which cuts straight through the middle of that region. Both are built on the sparse LU solver of the `natID` SDK, and both are implemented from scratch rather than called from a library.

The contrast between the two is the reason the visualization exists. A user types a small model as ordinary text, and the application draws the feasible region, animates the simplex pivoting from corner to corner, and overlays the interior point trajectory cutting through to the same optimum. For larger problems, where no picture is possible, a benchmark tab sweeps synthetic instances of growing size and charts how the two methods scale. This report covers the formulation, both algorithms and their mapping onto the `natID` sparse solver, the geometry and interface code, and the validation results.

2 Problem Formulation

Everything is solved in the standard form

$$\min c^T x \quad \text{subject to} \quad Ax = b, \quad x \geq 0, \quad A \in \mathbb{R}^{m \times n} \text{ sparse.}$$

The class `LPPProblem` converts the user's model into this form. A maximization becomes a minimization of $-c$, with the sign restored for display through `StandardLP::objSign`. A constraint $a^T x \leq b$ receives a slack variable with coefficient $+1$, and $a^T x \geq b$ receives a surplus variable with coefficient -1 . Non-negativity of the original variables is implicit.

The array `slackOfRow[i]` records which column is the $+1$ slack of row i . Those columns form an identity submatrix, giving a ready-made starting basis whenever $b \geq 0$, so phase I can be skipped entirely — the common case for the small inequality models typed into the 2D and 3D tabs. The matrix itself is stored column-wise in `SparseColMatrix` as lists of (row, value) pairs, which is both the access pattern the revised simplex needs, since it prices and solves with individual columns a_j , and what the SDK's `sparse::ISolver::addTriple` consumes.

3 Implementation and Architecture

3.1 The Revised Simplex Method

Let B be the index set of the basis, one column per row, and let $\mathcal{B} = A(:, B)$ be the corresponding square submatrix. Each iteration computes

$x_B = \mathcal{B}^{-1}b$	basic solution (FTRAN)
$y = \mathcal{B}^{-T}c_B$	simplex multipliers (BTRAN)
$d_j = c_j - y^T a_j$	reduced costs
$w = \mathcal{B}^{-1}a_q$	entering direction (FTRAN)
$\theta = \min\{x_B(i)/w_i : w_i > 0\}$	ratio test

Only the basis is ever factorized; the full tableau is never formed. That is the defining property of the *revised* method, and it is the reason the method pairs naturally with a sparse LU factorization.

3.2 Basis Factorization with the natID Solver

The class `BasisFactor` wraps the SDK. Each refactorization creates two fresh solvers through `sparse::createDblSolver`, one loaded with $\mathcal{B}$ and one with $\mathcal{B}^T$; two are needed because the `ISolver` interface exposes `solveExt` but neither a transpose solve nor a way to reset the values of an existing factorization. The diagonal pattern is seeded with `populateDiagonals(0.0)`, mirroring the SDK’s own matrix tests, after which `addTriple` accumulates each entry exactly once. Factorizing once per pivot rather than updating in place is the robust choice and puts the SDK solver to work on every iteration; production codes amortize the cost with eta-file or Forrest–Tomlin updates instead, which is recorded as future work.

3.3 Phase I with Minimal Artificial

Rows are first sign-normalized so that $b \geq 0$, with the flips recorded so the duals can be mapped back at the end. A row whose slack still carries coefficient +1 starts with that slack in the basis, and only the remaining rows receive an artificial variable with phase-I cost 1, so when no artificials are needed the phase-I loop never runs. If the phase-I objective stays positive beyond the feasibility tolerance, the problem is infeasible; otherwise any artificials left basic are driven out with degenerate pivots, one BTRAN per row and a sparse dot product per candidate column. A row for which no pivot exists is linearly dependent on the others: its artificial stays basic at zero and is never priced in phase II, which handles redundant constraints without a presolve step.

3.4 Pivot Rules and Termination

The entering variable is chosen by Dantzig’s rule, the most negative reduced cost. After a run of `blandAfter` consecutive degenerate pivots the code switches to Bland’s rule, taking the first improving index instead, which is provably immune to cycling. The leaving variable is chosen by the minimum ratio test, with ties broken by the largest $|w_i|$ for numerical stability. The fallback is not theoretical: the test suite runs Beale’s classic

cycling example, on which the plain Dantzig rule loops forever and the implementation terminates correctly.

3.5 The Interior Point Method

The interior point solver implements Mehrotra’s predictor–corrector method on the normal equations, for the primal–dual pair

$$(P) \min c^T x, Ax = b, x \geq 0 \quad (D) \max b^T y, A^T y + s = c, s \geq 0.$$

With $D^2 = \text{diag}(x/s)$ and the residuals $r_p = b - Ax$ and $r_d = c - A^T y - s$, each step solves

$$(AD^2A^T) \Delta y = r_p + AS^{-1}(Xr_d - r_{xs}), \quad \Delta s = r_d - A^T \Delta y, \quad \Delta x = S^{-1}(-X\Delta s - r_{xs}).$$

The affine pass uses $r_{xs} = XSe$, and the corrector reuses the same factorization with $r_{xs} = XSe - \sigma\mu e + \Delta X_{\text{aff}}\Delta S_{\text{aff}}e$ and $\sigma = (\mu_{\text{aff}}/\mu)^3$. One factorization therefore serves two solves per iteration. Steps take 0.99 of the distance to the boundary, and the starting point is Mehrotra’s, obtained from two solves with AA^T followed by a positivity shift.

The matrix AD^2A^T is symmetric positive definite, so the solver is created with **Symmetry::SymmetricPosDef** and only the lower triangle is inserted, the convention the SDK’s tests confirm. Its sparsity pattern is computed once, symbolically, from the column outer products $\sum_j d_j a_j a_j^T$, and only the values are refilled each iteration. A small diagonal regularization keeps the factorization stable, with a fallback to plain LU with mirrored entries if it ever fails.

3.6 Synthetic Instance Generator

Benchmark instances are generated together with both feasibility certificates, so an optimum is guaranteed to exist. A sparse A is built with a round-robin anchor that touches every row, the rest of each column filled at random. A strictly positive x^* gives $b = Ax^*$ (primal feasible), and a free y^* with a positive s^* gives $c = A^T y^* + s^*$ (dual feasible), so by strong duality the problem is feasible and bounded. The optimal value is deliberately not assumed to be $c^T x^*$, since the simplex usually finds a better vertex; correctness is checked through the KKT residuals and the gap between the two solvers instead.

3.7 Geometry for the Visualization

The feasible region is computed geometrically rather than inferred from the solver. In two dimensions the polygon comes from Sutherland–Hodgman clipping of a large box against every half-plane, an equality contributing both directions. In three dimensions vertices are enumerated over all triples of planes — user planes, axis planes, and a safety box capping otherwise unbounded regions — then filtered for feasibility and deduplicated, and each plane’s face is its incident vertices sorted by angle in the plane basis; the canvas renders faces back to front with the painter’s algorithm under a user-controlled yaw and pitch. The small 2×2 and 3×3 intersection systems behind this use the SDK’s **dense::Matrix**, the one place in the project where a dense solve is genuinely the right tool.

3.8 MPS Export and the External Comparison

The module `MPS.h` writes the fixed-field `NAME/ROWS/COLUMNS/RHS/ENDATA` layout that COIN-OR CLP accepts and reads the same subset back, converting slacks and surpluses on input and rejecting `RANGES` and `BOUNDS` explicitly rather than misreading them. Every benchmark instance is exported, and `scripts/run_clp.sh` runs CLP twice per file, primal simplex and barrier, collecting times and objectives into a CSV — so this implementation and an established external solver are measured on identical inputs.

3.9 Graphical Interface

The interface is a tabbed window built on the `natID` GUI classes: a 2D tab splitting the model editor and controls on the left from the polygon canvas on the right, a 3D tab with the same layout and a rotatable polytope canvas, and a benchmark tab with sweep parameters beside two chart canvases. The class `VizModel` owns the parsed model, the standard-form problem, both solver results with their recorded paths, the computed geometry and the animation cursor; the canvases only read from it. `LinearExprParser` accepts ordinary lines such as `3x+2y<=18`, with variables named either `x,y,z` or `x1..x3`, expressions on both sides, and comments.

Animation is driven by a `gui::Timer` on the tab view, and the Run menu and toolbar route through `MainWindow::onActionItem` into the active tab. The benchmark sweep runs on a separate `std::thread` and marshals progress back with `gui::thread::asyncExecInMainThread`, so widgets are only ever touched from the main thread; its charts use a small custom canvas with nice axis ticks, multiple series and a legend. Because both solvers record a snapshot of every iteration — number, phase, entering and leaving indices, objective value and full iterate — the pivot animation and the interior point trajectory replay from identical data structures.

3.10 Resources, Translations and Build

All interface text is loaded through the `natID` resource system: `res/DevRes.xml` declares the artwork and translation configurations, `res/main.xml` defines the interface, and translations in `res/tr/EN` and `res/tr/BA` ship the application in English and Bosnian with a settings dialog for switching.

The repository follows the required submission layout: `Docs/` holds this report and `Implementation/` everything needed to build — `src/core/` the two solvers, `src/viz/` the tabs and canvases, `src/bench/` the benchmark, `cli/` the console tool, `tests/` the suite and `installer/` the packaging configuration. CMake builds the `simplexLP` GUI and the `lpBench` console benchmark, and since every path resolves relative to the CMake list directory the project builds unchanged on Windows, macOS and Linux, with installers for all three produced by the SDK's `SetupCollector`.

4 Validation

The suite in `tests/standalone/` compiles the unmodified solver headers against shims for the SDK interfaces: a dense LU with the same accumulate and one-triangle-for-symmetric semantics as `sparse::ISolver`, plus stand-ins for `dense::Matrix` and `cnt::`

SafeFullVector. The algorithms are therefore tested exactly as shipped, with no test-only modifications. Its 86 checks cover:

- Dantzig’s production example, which reaches the optimum 36 at $(2, 6)$ with a crash basis and therefore zero phase-I iterations, with a monotone objective along the path;
- a $\geq$ model that exercises phase I, and an equality model;
- explicit detection of infeasible and of unbounded problems;
- a degenerate vertex, and Beale’s cycling example, which terminates thanks to the Bland fallback;
- duplicated equality rows, checking the redundancy handling;
- an MPS write and read round trip reproducing the instance exactly;
- exact vertex and face counts for the 2D polygon and for a 3D cube and tetrahedron;
- eight random instances up to $m = 60$, $n = 150$, where both methods report optimality and agree to within 10^{-8} , with the KKT residuals passing.

On those random instances the simplex iteration count climbs from 10 to 182 as the problems grow, while the interior point method stays between 7 and 11 throughout. That is the textbook contrast between the two families of methods, and the benchmark tab reproduces it at larger scale with the real natID factorizations. In addition, the complete application and console tool are syntax-checked against the real SDK headers, so every call into natID is verified independently of the shims.

5 Conclusion

The project implements both major families of linear programming algorithms on a shared sparse-linear-algebra foundation and makes the difference between them visible: the revised simplex factorizes only the basis and moves along the boundary, the interior point method factorizes the normal equations and moves through the interior, and the same window shows both converging to the same optimum on the same model. The benchmark and MPS export extend the comparison beyond what can be drawn, including against an external solver on identical inputs. The natural extensions are recorded in the implementation notes: Forrest–Tomlin basis updates with periodic refactorization, steepest-edge or Devex pricing, the Harris two-pass ratio test, a homogeneous self-dual interior point formulation that could also certify infeasibility, a presolve stage, and full MPS bounds support so the Netlib problems could be read directly.

The source code of the project can be found at https://github.com/ssijaric1/ProjNO_SimplexL_Sijaric.